Course: Malware Analysis (CIP-A106)

Student Name: Habeebah Adeyinka Olugbogi

Registration Number: c11.ehit2617337

Submission Date: 28th of September, 2026

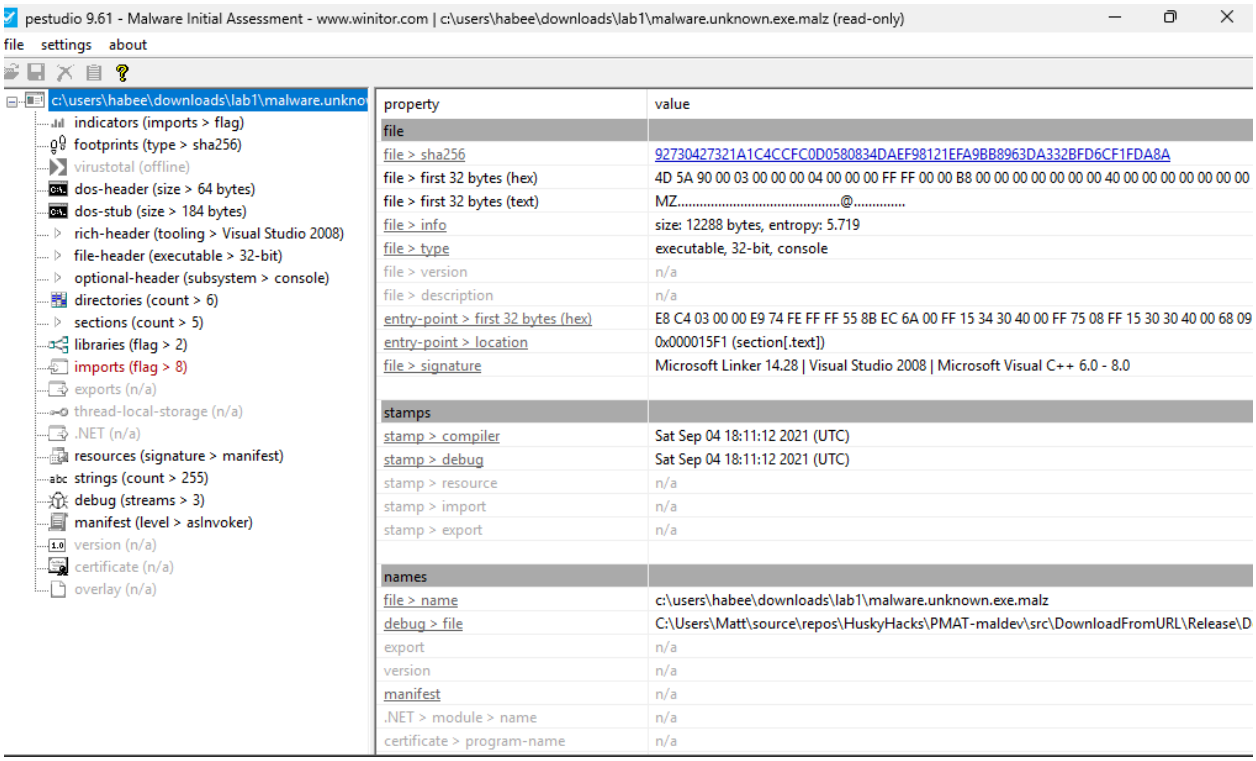
EXECUTIVE SUMMARY

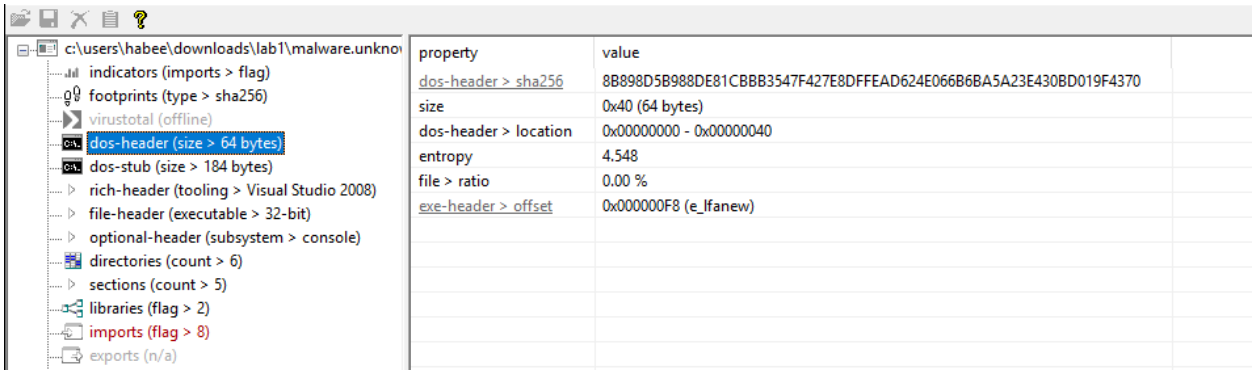
The sample is a Windows console program. It is 12,288 bytes and it is 32-bit. In its imports, there are calls for web use. These include InternetOpenW and InternetOpenUrlW. It also uses URLDownloadToFileW to get a file. For launching, it calls CreateProcessW and ShellExecuteW. In the strings, there are two URLs. The debug path includes the folder name DownloadFromURL. On VirusTotal, 56 out of 70 antivirus engines mark it. The static signals suggest a downloader. Still, those signs by themselves do not tell what it does at runtime.

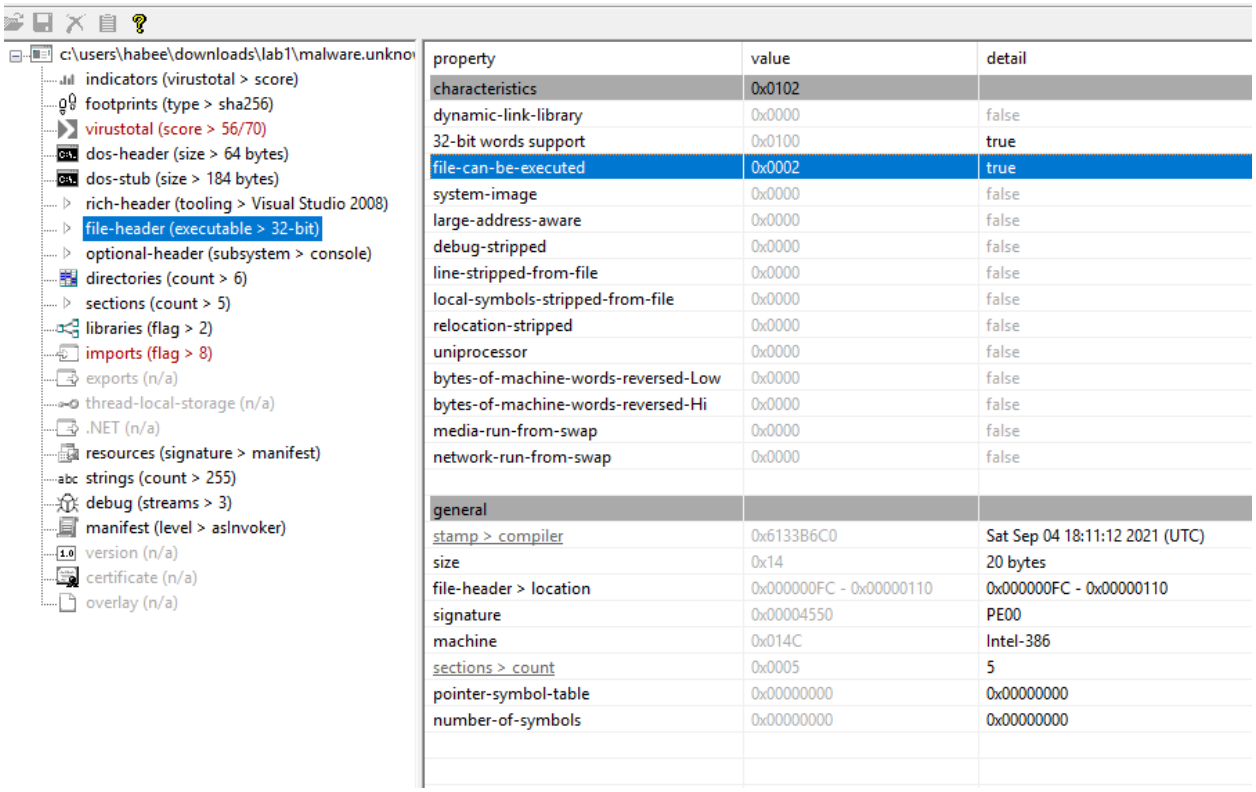
Task A: Basic File Examination

File type

Item	Finding
File type (pestudio)	Executable, 32-bit, console
Machine:	Intel-386 (0x014C)
Signature:	PE00
First bytes:	4D 5A ("MZ")
File size:	12,288 bytes
Subsystem:	Console







Hashes

Hash	**Value**
MD5	1D8562C0ADCAEE734D63F7BAACA02F7C
SHA-256	———————92730427321A1C4CCFC0D0580834DAEF98121EFA9BB8963DA332BFD6CF1FDA8A
IMPHASH (MD5)	F2D1B81B70ADF3F2DCCC6D462AE64DC4

The SHA-256 from PowerShell matches the SHA-256 shown in pestudio.

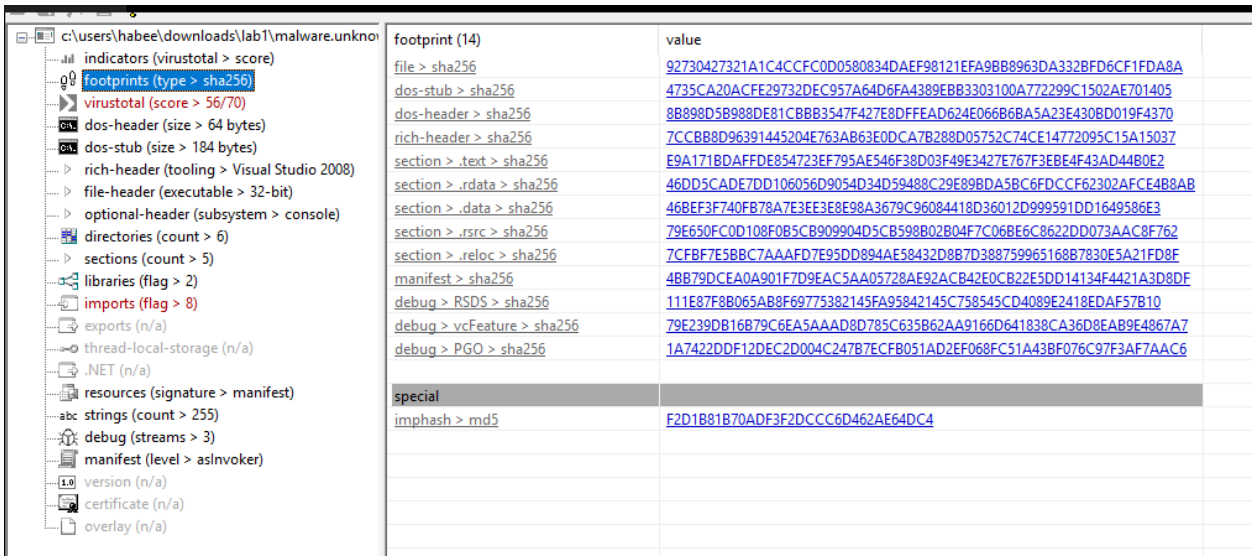
```
PS C:\Users\habee\Downloads> cd ~\Downloads\LAB1
PS C:\Users\habee\Downloads\LAB1> Get-FileHash .\Malware.Unknown.exe.malz -Algorithm SHA256

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          92730427321A1C4CCFC0D0580834DAEF98121EFA9BB8963DA332BFD6CF1FDA8A       C:\Users\habee\Downloads\LAB1\Malw...


PS C:\Users\habee\Downloads\LAB1> Get-FileHash .\Malware.Unknown.exe.malz -Algorithm MD5

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
MD5             1D8562C0ADCAEE734D63F7BAACA02F7C                                       C:\Users\habee\Downloads\LAB1\Malw...


PS C:\Users\habee\Downloads\LAB1>
```



Entropy

Part	**Entropy**
Whole file	5.719
DOS header	4.548
DOS stub	5.021
.text section	6.506
.rdata section	4.423
.data section	0.321

property	value
dos-header > sha256	8B898D5B988DE81CBBB3547F427E8DFFEAD624E066B6BA5A23E430BD019F4370
size	0x40 (64 bytes)
dos-header > location	0x00000000 - 0x00000040
entropy	4.548
file > ratio	0.00 %
exe-header > offset	0x000000F8 (e_lfanew)

property	value	value	value
section	section[0]	section[1]	section[2]
name	.text	.rdata	.data
section > sha256	E9A171BDAFFDE854723EF79...	46DD5CADE7DD106056D905...	46BEF3F740FB78A7E3E
entropy	6.506	4.423	0.321
file > ratio (91.67%)	45.83 %	33.33 %	4.17 %
raw-address (begin)	0x00000400	0x00001A00	0x00002A00
raw-address (end)	0x00001A00	0x00002A00	0x00002C00
raw-size (11264 bytes)	0x00001600 (5632 bytes)	0x00001000 (4096 bytes)	0x00000200 (512 bytes)

property	value
dos-stub > sha256	4735CA20ACFE29732DEC957A64D6FA4389EBB3303100A772299C1502AE701405
dos-stub > location	0x00000040 - 0x000000F8
size	0xB8 (184 bytes)
entropy	5.021
file > ratio	1.50 %
first 32 bytes (hex)	0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 69 73 20 70 72 6F 67 72 61 6D 20...
first 32 bytes (hex)	.....!....L..!This program canno
message	!This program cannot be run in DOS mode.

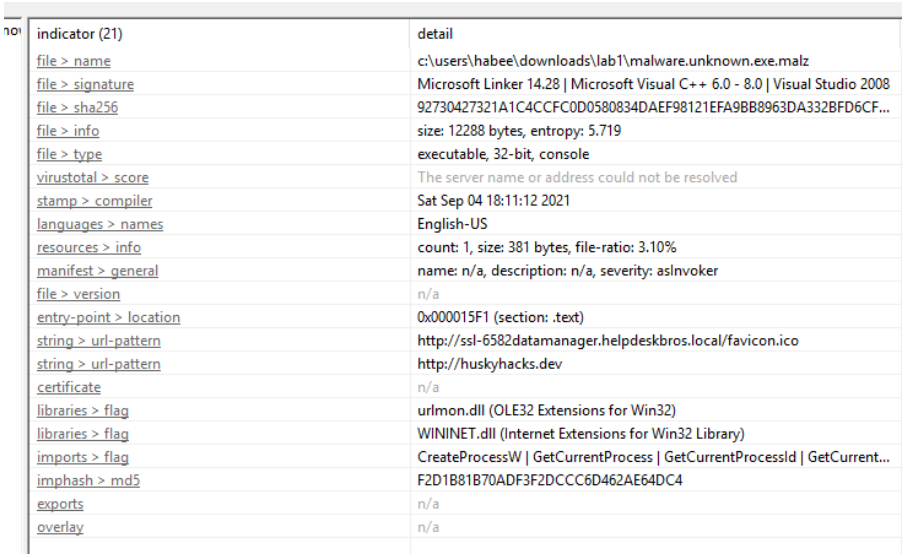
property	value
dos-header > sha256	8B898D5B988DE81CBBB3547F427E8DFFEAD624E066B6BA5A23E430BD019F4370
size	0x40 (64 bytes)
dos-header > location	0x00000000 - 0x00000040
entropy	4.548
file > ratio	0.00 %
exe-header > offset	0x000000F8 (e_lfanew)

Strings

pestudio reports more than 255 strings. Here are some:

Type	Finding
URL	http://ssl-6582datamanager.helpdeskbros.local/favicon.ico
URL	http://huskyhacks.dev
IP addresses	None seen in my screenshots
Windows APIs	It's in theApi_list.txt file

File path	Debug path
C:\Users\Matt\source\repos\HuskyHacks\PMAT-maldev\src\DownloadFromURL\Release\D...	



indicator (21)	detail
file > name	c:\users\habee\downloads\lab1\malware.unknown.exe.malz
file > signature	Microsoft Linker 14.28 | Microsoft Visual C++ 6.0 - 8.0 | Visual Studio 2008
file > sha256	92730427321A1C4CCFC0D0580834DAEF98121EFA9BB8963DA332BFD6CF...
file > info	size: 12288 bytes, entropy: 5.719
file > type	executable, 32-bit, console
virustotal > score	The server name or address could not be resolved
stamp > compiler	Sat Sep 04 18:11:12 2021
languages > names	English-US
resources > info	count: 1, size: 381 bytes, file-ratio: 3.10%
manifest > general	name: n/a, description: n/a, severity: asInvoker
file > version	n/a
entry-point > location	0x000015F1 (section: .text)
string > url-pattern	http://ssl-6582datamanager.helpdeskbros.local/favicon.ico
string > url-pattern	http://huskyhacks.dev
certificate	n/a
libraries > flag	urlmon.dll (OLE32 Extensions for Win32)
libraries > flag	WININET.dll (Internet Extensions for Win32 Library)
imports > flag	CreateProcessW | GetCurrentProcess | GetCurrentProcessId | GetCurrent...
imphash > md5	F2D1B81B70ADF3F2DCCC6D462AE64DC4
exports	n/a
overlay	n/a

Packing assessment

Entropy looks mid range. The top value is 6.506 for .text. Section list seems ordinary. I see .text, .rdata, .data, .rsrc, and .reloc. For .text, the raw size is 5,632 bytes and the virtual size is 5,537 bytes. Those two numbers are pretty close. The import table can be read. It shows 52 imports from 11 libraries.

Overall, I do not spot a clear sign of packing. My confidence is only medium since I rely on pestudio output.

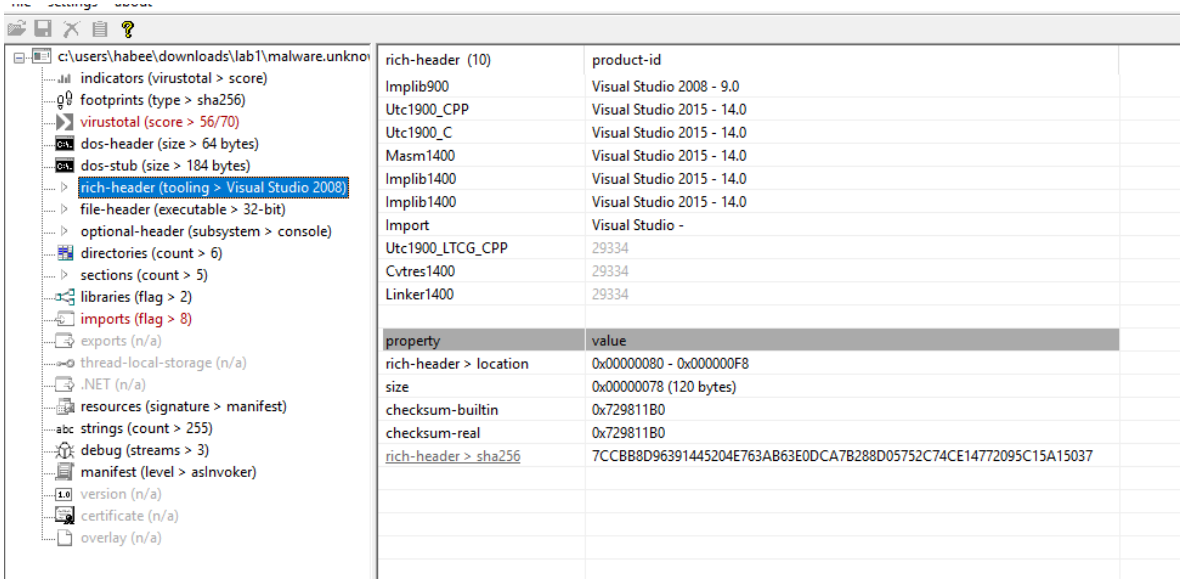
Task B: PE layout and import check

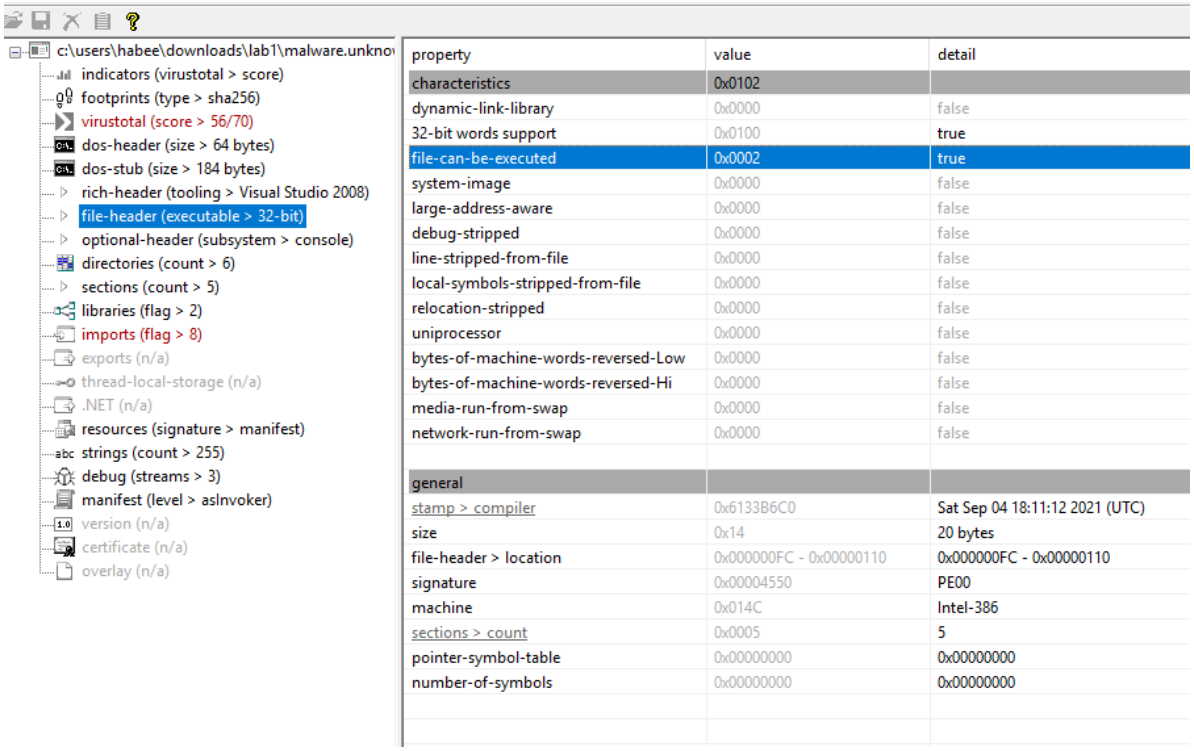
Build time

Item	**Value**
Compiler stamp	Sat Sep 04 18:11:12 2021 (UTC), 0x6133B6C0
Debug stamp	Sat Sep 04 18:11:12 2021 (UTC)
Linker	Microsoft Linker 14.28

The compiler stamp and the debug stamp match, so the file keeps one consistent date. Still, timestamps can be changed later, so I do not treat that time as solid proof.

The rich header shows Visual Studio 2008 under Implib900. It also shows Visual Studio 2015. In pestudio, the tool name is shown as “Visual Studio 2008”. The rich header checksum values line up too. They are 0x729811B0 for both the built-in and the real values.



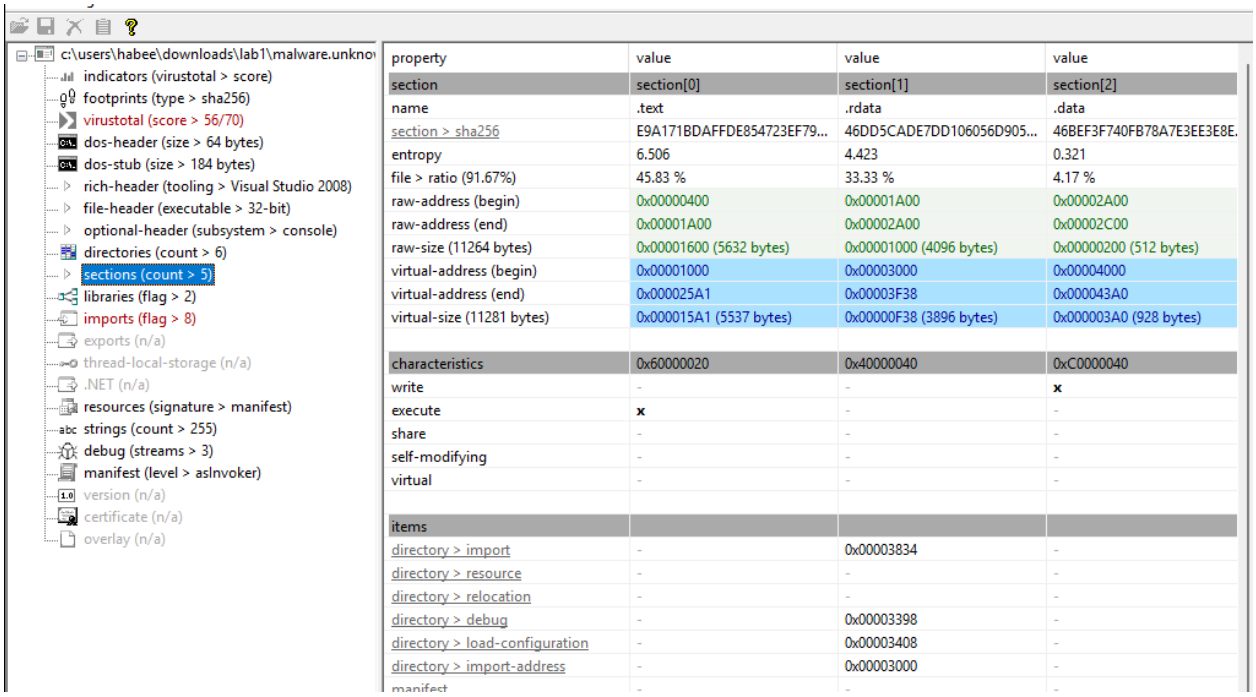


SECTION HEADERS

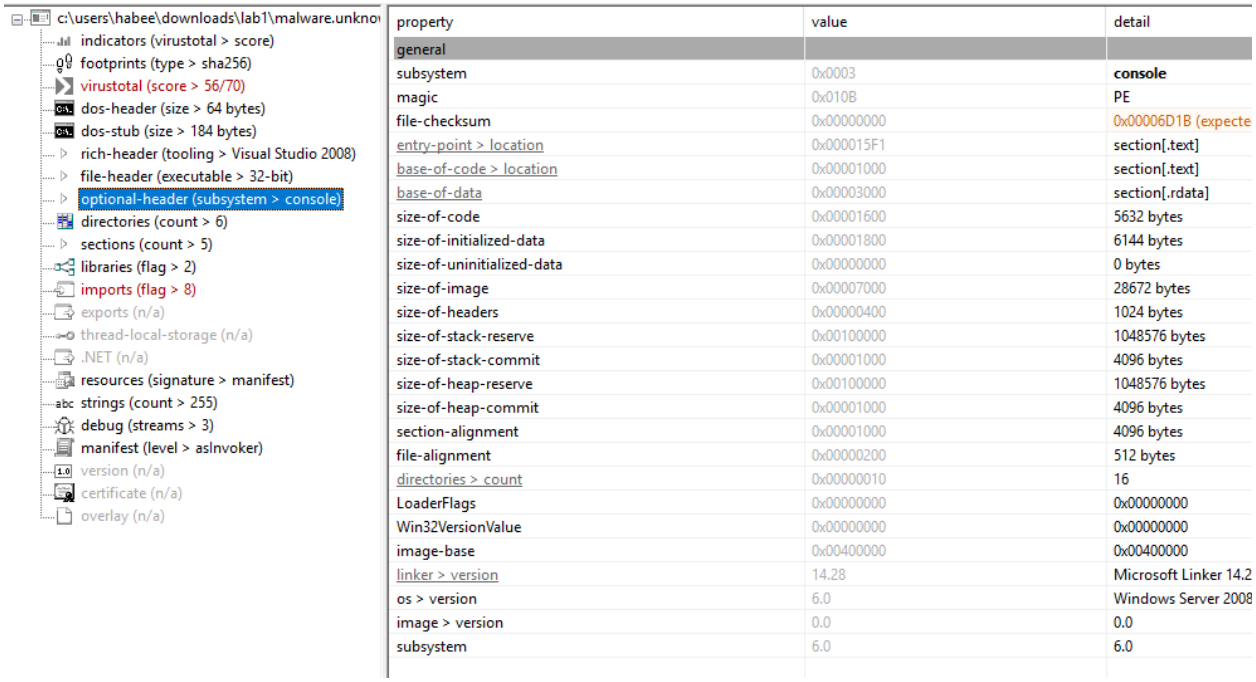
The only executable part is .text. Only .data can be written to. I did not find any section that has both traits. I also did not see an executable section for data.

In .data, the virtual size is 928, but the raw size is 512.

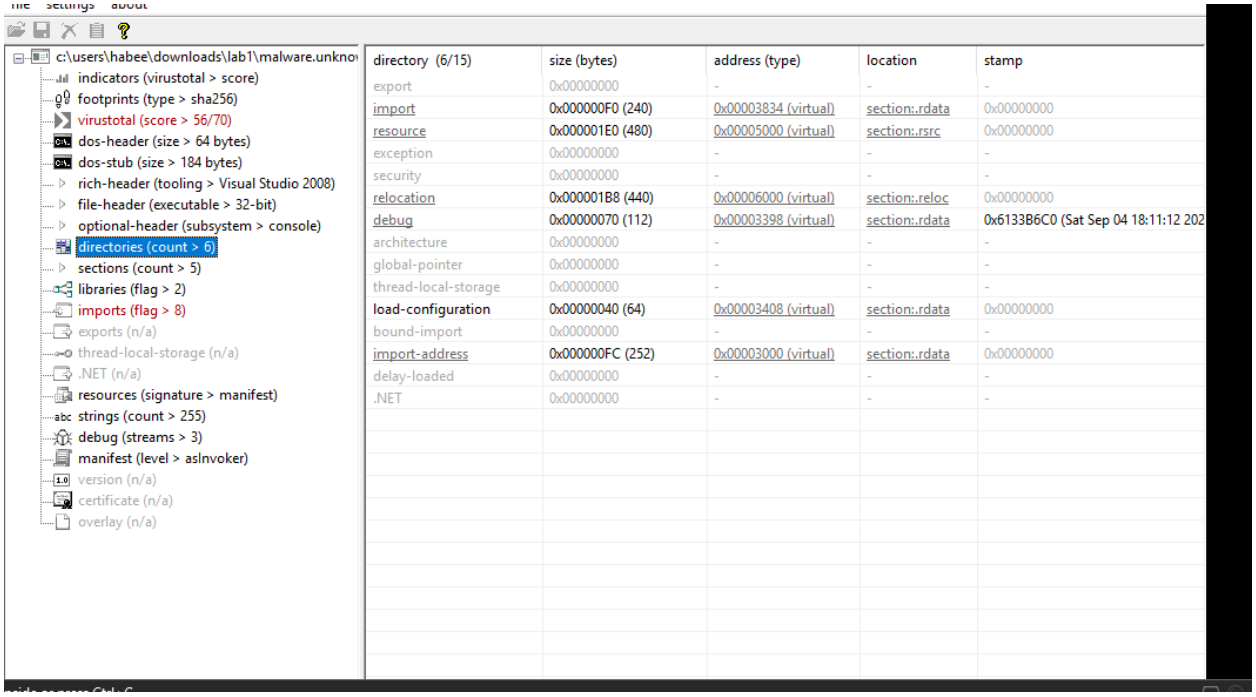
Header details: the entry point is 0x000015F1 and it sits in .text. The checksum in the file header is 0x00000000. In pestudio, the orange “expected” value is 0x00006D1B. The section alignment is 0x1000, and the file alignment is 0x200.



property	value	value	value
section	section[0]	section[1]	section[2]
name	.text	.rdata	.data
section > sha256	E9A171BDAFFDE854723EF79...	46DD5CADE7DD106056D905...	46BEF3F740FB78A7E3EE3E8E.
entropy	6.506	4.423	0.321
file > ratio (91.67%)	45.83 %	33.33 %	4.17 %
raw-address (begin)	0x00000400	0x00001A00	0x00002A00
raw-address (end)	0x00001A00	0x00002A00	0x00002C00
raw-size (11264 bytes)	0x00001600 (5632 bytes)	0x00001000 (4096 bytes)	0x00000200 (512 bytes)
virtual-address (begin)	0x00001000	0x00003000	0x00004000
virtual-address (end)	0x000025A1	0x00003F38	0x000043A0
virtual-size (11281 bytes)	0x000015A1 (5537 bytes)	0x00000F38 (3896 bytes)	0x000003A0 (928 bytes)
characteristics	0x60000020	0x40000040	0xC0000040
write	-	-	x
execute	x	-	-
share	-	-	-
self-modifying	-	-	-
virtual	-	-	-
items			
directory > import	-	0x00003834	-
directory > resource	-	-	-
directory > relocation	-	-	-
directory > debug	-	0x00003398	-
directory > load-configuration	-	0x00003408	-
directory > import-address	-	0x00003000	-
manifest	-	-	-



property	value	detail
general		
subsystem	0x0003	**console**
magic	0x010B	PE
file-checksum	0x00000000	0x00006D1B (expecte
entry-point > location	0x000015F1	section[.text]
base-of-code > location	0x00001000	section[.text]
base-of-data	0x00003000	section[.rdata]
size-of-code	0x00001600	5632 bytes
size-of-initialized-data	0x00001800	6144 bytes
size-of-uninitialized-data	0x00000000	0 bytes
size-of-image	0x00007000	28672 bytes
size-of-headers	0x00000400	1024 bytes
size-of-stack-reserve	0x00100000	1048576 bytes
size-of-stack-commit	0x00001000	4096 bytes
size-of-heap-reserve	0x00100000	1048576 bytes
size-of-heap-commit	0x00001000	4096 bytes
section-alignment	0x00001000	4096 bytes
file-alignment	0x00000200	512 bytes
directories > count	0x00000010	16
LoaderFlags	0x00000000	0x00000000
Win32VersionValue	0x00000000	0x00000000
image-base	0x00400000	0x00400000
linker > version	14.28	Microsoft Linker 14.2
os > version	6.0	Windows Server 2008
image > version	0.0	0.0
subsystem	6.0	6.0



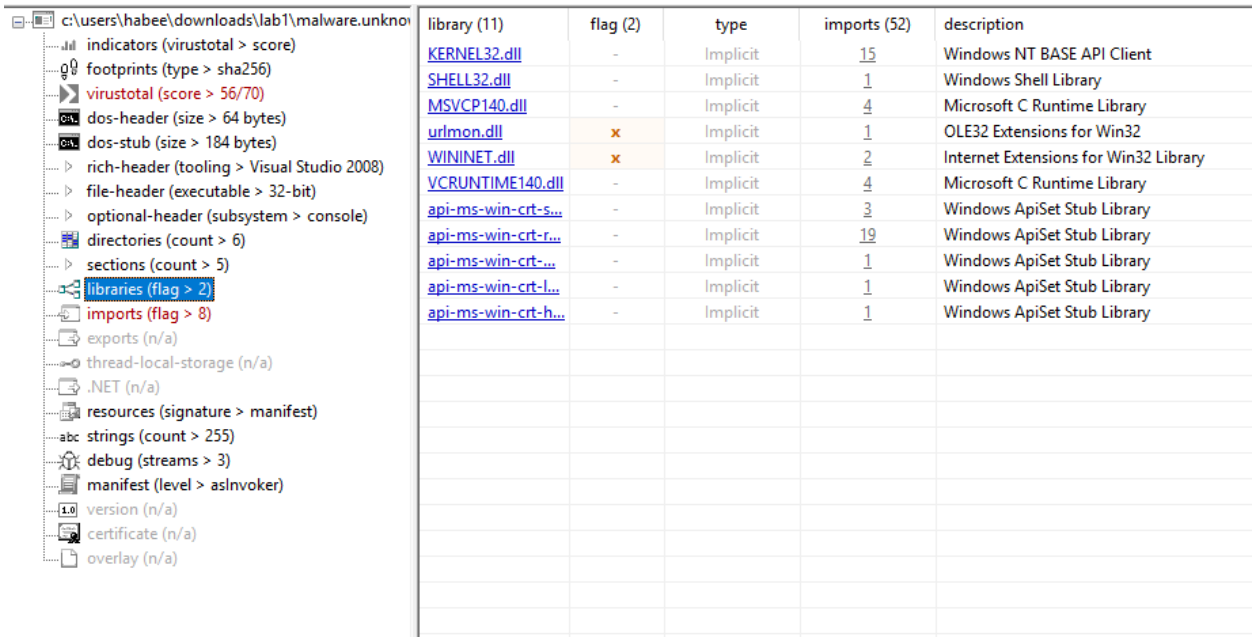
Imports (IAT)

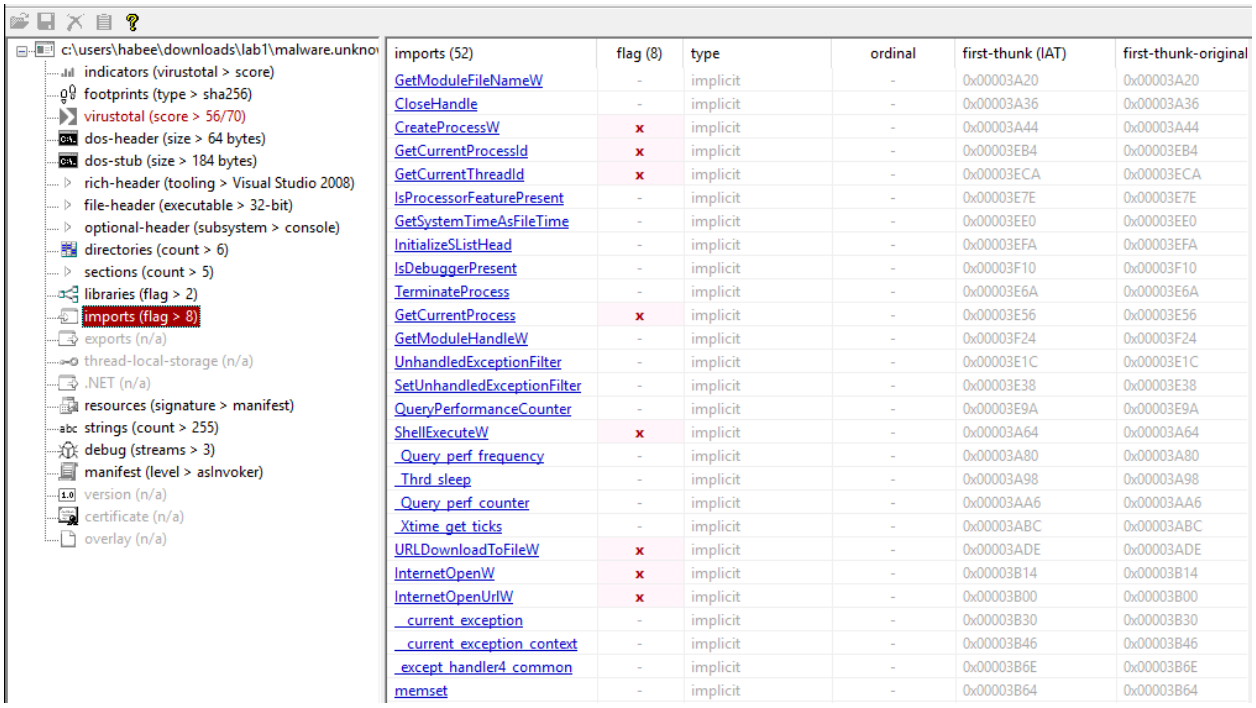
pestudio shows 52 imports from 11 libraries. Two libraries are flagged: urlmon.dll and wininet.dll.

Library	Imports	Flagged
KERNEL32.dll	15	No
SHELL32.dll	1	No
MSVCP140.dll	4	No
urlmon.dll	1	Yes
WININET.dll	2	Yes
VCRUNTIME140.dll	4	No
api-ms-win-crt-* (5 libraries)	3, 19, 1, 1, 1	No

Flagged imports (8): CreateProcessW, GetCurrentProcessId, GetCurrentThreadId, GetCurrentProcess, ShellExecuteW, URLDownloadToFileW, InternetOpenW, InternetOpenUrlW.

Some other imports are: GetModuleFileNameW, CloseHandle, IsProcessorFeaturePresent, GetSystemTimeAsFileTime, InitializeSListHead, IsDebuggerPresent, TerminateProcess, GetModuleHandleW, UnhandledExceptionFilter, SetUnhandledExceptionFilter, QueryPerformanceCounter, _Query_perf_frequency, _Thrd_sleep, _Query_perf_counter, _Xtime_get_ticks, ___current_exception, ___current_exception_context, _except_handler4_common, memset.





Export table

pestudio shows exports as n/a. There is no export table. I found no sign of reflective loading or unusual exports.

Resources and TLS

Item	Finding
Resources	1 resource, 381 bytes (3.10% of file), type manifest
Manifest level	asInvoker
Icons	None seen
Embedded binaries	None seen
TLS callbacks	Thread-local-storage shows n/a. None found
.NET, certificate, version info, overlay	All n/a

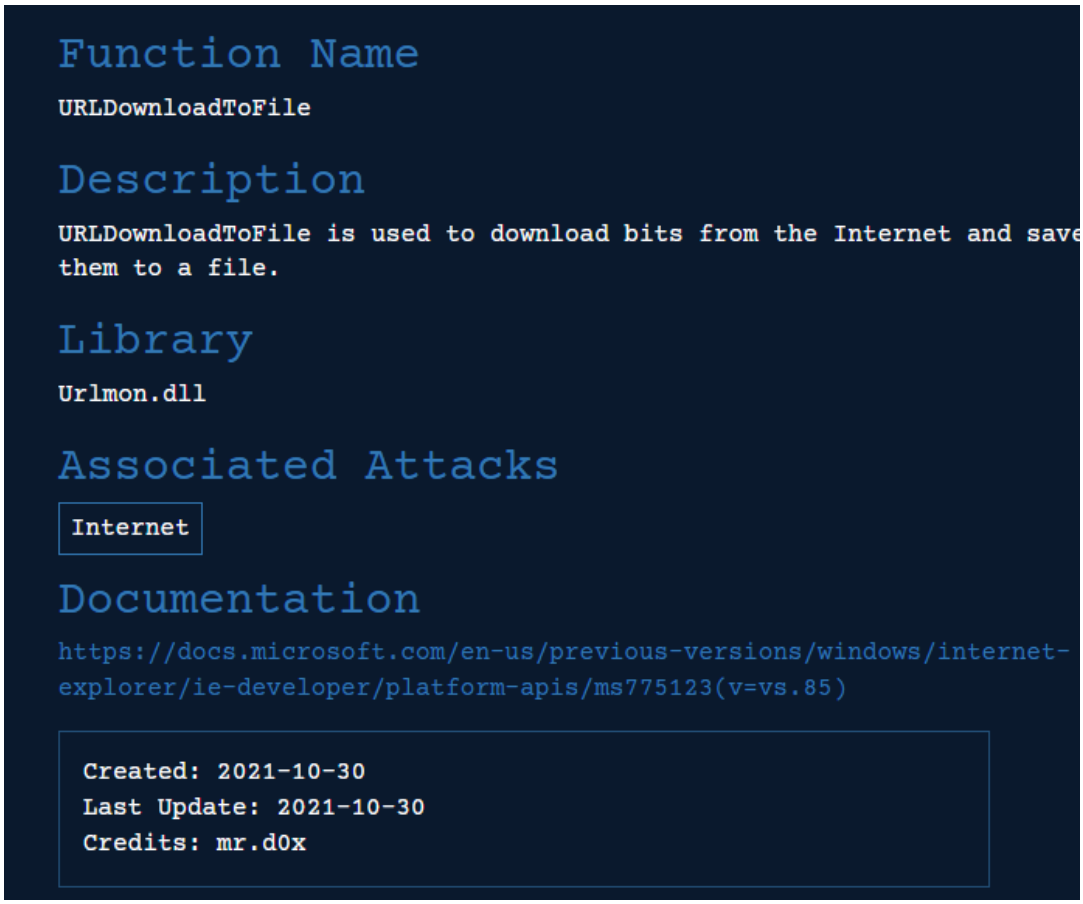
Task C: API Intelligence Using MalAPI.io

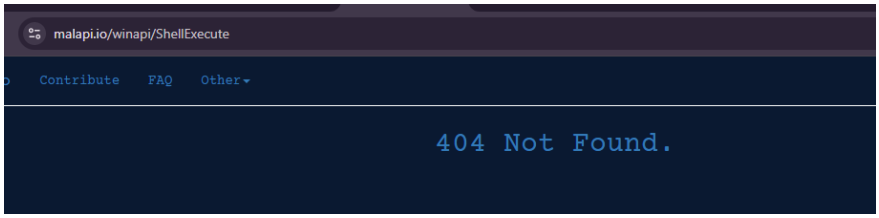
The MalAPI pages I captured show only the description, DLL and "Associated Attacks" category. They show no prevalence percentage. I cannot say any API is above 60%.

Native API (NTDLL)

No ntdll.dll appears in the library list, and no native API appears in the imports I can see. No native API findings.

API scorecard





Function Name

IsDebuggerPresent

Description

IsDebuggerPresent is used to determine whether the calling process is being debugged by a user-mode debugger.

Library

Kernel32.dll

Associated Attacks

Anti-Debugging

Documentation

https://docs.microsoft.com/en-us/windows/win32/api/debugapi/nf-debugapi-isdebuggerpresent

Created: 2021-10-30
Last Update: 2021-10-30
Credits: mr.d0x

Function Name

ShellExecuteA

Description

ShellExecuteA is used to perform an operation on a specified file.

Library

Shell32.dll

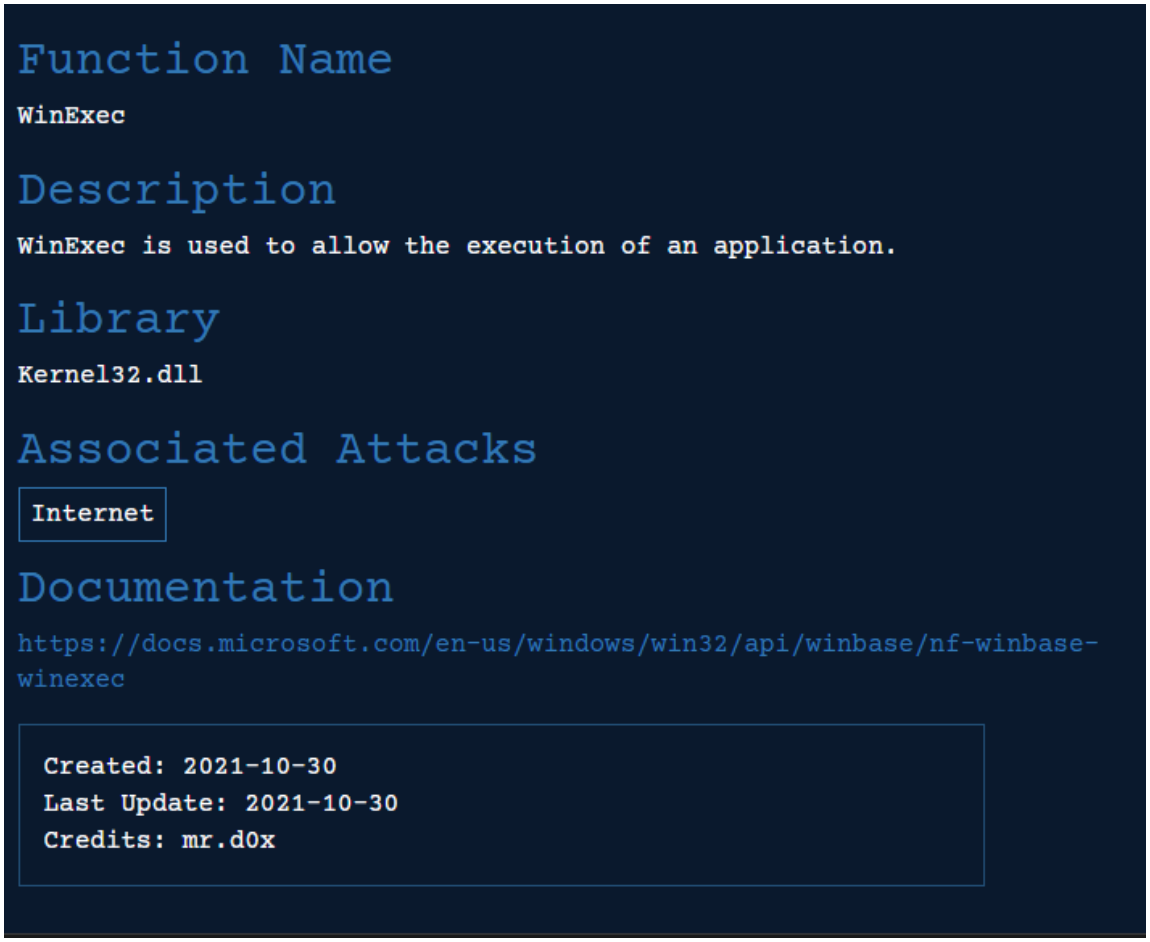
Associated Attacks

Internet

Documentation

https://docs.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecutea

Created: 2021-10-30
Last Update: 2021-10-30
Credits: mr.d0x



API Clusters

I reviewed the import list for API sets that are commonly tied to process injection, persistence, anti analysis, network activity, and program execution. In what I could see, I did not spot process injection related APIs. I also did not see registry, service, or scheduled task related imports that would suggest persistence.

For anti analysis, I only saw IsDebuggerPresent. The network part shows three imports. They are InternetOpenW, InternetOpenUrlW, and URLDownloadToFileW. For launching work, I saw two imports. Those are CreateProcessW and ShellExecuteW.

One limit matters here. I could not view 25 of the 52 imports in my screenshots. So these findings apply only to the imports that were visible.

Strange Pairing, API Hashing, COM/WMI

In this case, the key pairing is the download APIs plus the execution APIs. Being able to pull a file from the internet and also start a program is notable. Those abilities can be combined in the

same workflow. I did not find API hashing in the screenshots I had. I also did not see COM, WMI, or scheduled task related APIs.

Possible Call Chain

From the imported APIs, the visible strings, and the debug folder path, one possible flow is like this. First, it starts an internet session with InternetOpenW. Next, it pulls content with InternetOpenUrlW or URLDownloadToFileW. After that, it triggers execution with ShellExecuteW or CreateProcessW.

Even so, this is only about possible actions. The files I saw do not prove the call order. They also do not prove that the network and execute steps really ran. I also could not tell what it would download or what it would launch. Because of that, my confidence in this flow is weak to fair.

Behavioural Reconstruction

Abilities

It looks like delivery from a URL can happen. I found URLDownloadToFileW and InternetOpenUrlW in the imports. There are URL strings too. The debug path has a folder name that points to DownloadFromURL. For command and control, the proof is thin. I noticed two URLs, but there is nothing clear that shows two way control. I also did not find signs of data leaving the host. I did not see persistence. I did not see privilege escalation either. The requested execution level is asInvoker, so it does not request higher rights.

Anti-Analysis

The sample imports IsDebuggerPresent. MalAPI also tags it as Anti-Debugging. Still, my current evidence does not show how it is used to hinder analysis. This call can show up in regular software as well. I did not notice other anti analysis tricks in what I reviewed.

VirusTotal (Supporting Evidence)

VirusTotal shows a score of 56/70, from a scan on 2026-09-20. Some of the labels I saw are Trojan.Bulz, TrojanDownloader:Win32/SelfDel (Alibaba), Trojan/Win32.SelfDel (Antiy-AVL), Win32/TrojanDownloader.Small.BKM (ESET), Dropper.DR/W32.MalwareX (F-Secure) and Trojan.MulDrop (DrWeb). Some engines showed undetected, for example APEX, Acronis and ClamAV. These are labels from the antivirus vendors, and I did not confirm them myself.

MITRE ATT&CK mapping

Technique	Evidence	Confidence
T1105 Ingress Tool Transfer	URLDownloadToFileW, InternetOpenUrlW, URL strings	Medium
T1071.001 Application Layer Protocol: Web Protocols	wininet.dll imports, HTTP URLs	Low
T1106 Native API	CreateProcessW, ShellExecuteW	Low to medium
T1622 Debugger Evasion	IsDebuggerPresent import	Low

I found no evidence for other techniques.

Vulnerability exploitation

No strings, imports or metadata suggest exploitation.

Conclusion

The evidence points to a small 32 bit downloader. It uses internet and download functions. It also calls program launch routines. There are two URLs in the file. I see a debug route called DownloadFromURL.

I did not find packing. I did not find export activity. I did not find TLS callback use. I also saw noclear persistence. I saw no injection steps. I saw no clear data exfiltration either.

There is only one anti debug function noted. I still need to check it in more detail.

There are limits to this look. The sample was not executed. There are 25 imported items that were not observed. MalAPI also did not provide any usage numbers.